Bài tập lớn A1: CNN và Transformer trên Food-101

Mốc A1.1: tập dữ liệu và giao thức thí nghiệm

CO5085 Học sâu và ứng dụng trong thị giác máy tính, học kỳ 261 (2026–2027)

Huỳnh Văn Tâm Thiện (2470094)
Lê Hữu Trọng (2470578)

Giảng viên: TS. Lê Thành Sách, TS. Nguyễn Quốc Minh
Trường Đại học Bách khoa, ĐHQG-HCM

21/10/2026

Mục lục

1 Tóm tắt và đối chiếu với yêu cầu của đề **2**

2 Mục tiêu và câu hỏi nghiên cứu **2**

3 Tập dữ liệu **3**

4 Chia train, validation, test **4**

5 Khảo sát dữ liệu **4**

6 Tiền xử lý và tăng cường dữ liệu **6**

7 Mô hình **7**

8 Chế độ huấn luyện **10**

9 Giao thức thí nghiệm **10**

10 Chiến lược đánh giá **11**

11 Hạ tầng, tái lập và ngân sách **14**

12 Kế hoạch, rủi ro và hạn chế **14**

1 Tóm tắt và đối chiếu với yêu cầu của đề

Báo cáo này nộp cho mốc A1.1 (tập dữ liệu và giao thức thí nghiệm), nên chưa có kết quả huấn luyện. Các quyết định đã chốt:

- **Tập dữ liệu:** Food-101, 101 lớp; train, validation, test gồm 68 175, 7 575 và 25 250 ảnh (Mục 4).
- **Mô hình:** ResNet-50 (CNN) và DeiT-S (Transformer), cùng pretrain trên ImageNet-1k.
- **Chế độ huấn luyện:** từ đầu (scratch), chỉ train head, đóng băng nửa đầu (partial) và finetune toàn bộ (full); tổng cộng 8 run với seed 42.
- **Dữ liệu vào mô hình:** một pipeline tiền xử lý và tăng cường duy nhất, dùng chung cho mọi mô hình và mọi chế độ.

Bảng 1 chỉ ra mục nào của báo cáo đáp ứng từng yêu cầu của đề.

Bảng 1: Đối chiếu yêu cầu của đề (mục 3.1, 4, 5 trong đề) với nội dung báo cáo.

Yêu cầu của đề	Cách báo cáo đáp ứng	Vị trí
Chọn tập ảnh công khai, tối thiểu 10 lớp, train từ vài nghìn ảnh; không dùng MNIST, Fashion-MNIST, CIFAR-10	Food-101: 101 lớp, 68 175 ảnh train, công khai, không thuộc ba tập trên	Mục 3
Nêu rõ giấy phép	không có giấy phép chuẩn; điều khoản *scientific fair use* và nguồn Foodspotting	Mục 3.2
Nêu rõ lý do chọn	bốn tiêu chí, kèm các tập đã loại và lý do	Mục 3.3
Mô tả split	test chính thức; validation tách phân tầng từ train; cùng split cho mọi run	Mục 4
Mô tả augmentation	bảng pipeline train và đánh giá, chuẩn hoá	Mục 6
Liệt kê CNN và Transformer sẽ dùng	ResNet-50 và DeiT-S: kiến trúc, tham số, chi phí	Mục 7
Kế hoạch scratch, pretrained, freeze	bốn chế độ, định nghĩa đóng băng, 8 run, siêu tham số	Mục 8, 9
Bổ sung ngoài yêu cầu của A1.1	chiến lược đánh giá, hạ tầng và ngân sách, kế hoạch, rủi ro	Mục 10–12

2 Mục tiêu và câu hỏi nghiên cứu

A1 so sánh một mạng tích chập (CNN) và một Transformer thị giác (ViT) trên một tập ảnh đủ lớn và đủ khó để khác biệt giữa hai họ kiến trúc, và giữa các cách dùng trọng số pretrained, có ý nghĩa. Bài chọn **Food-101** (101 món ăn, 101 000 ảnh), **ResNet-50** cho họ CNN và **DeiT-S** cho họ Transformer. Mỗi mô hình được huấn luyện theo bốn chế độ: từ đầu (*scratch*), và ba cách dùng trọng số pretrained ImageNet-1k là chỉ train head (*head*), đóng băng nửa đầu mạng (*partial*) và finetune toàn bộ (*full*).

Bốn câu hỏi A1 cần trả lời, kèm điều chúng tôi kỳ vọng trước khi chạy:

1. **Train từ đầu.** Với khoảng 68 000 ảnh train, CNN hay ViT tốt hơn? Kỳ vọng: ResNet-50 hơn DeiT-S, vì phép tích chập mang sẵn giả định cục bộ và bất biến tịnh tiến, còn ViT phải học các quan hệ đó từ dữ liệu [3, 4].
2. **Giá trị của pretrain.** Pretrain giúp mỗi họ bao nhiêu? Kỳ vọng: khoảng cách giữa scratch và full finetune của DeiT-S lớn hơn của ResNet-50.

3. **Khi nào đóng băng có lợi.** Đóng băng giảm chi phí và số tham số phải học; câu hỏi là mất bao nhiêu accuracy so với full finetune, và mức mất đó khác nhau thế nào giữa hai họ.
4. **Chi phí.** Thời gian mỗi epoch, bộ nhớ GPU đỉnh, số tham số trainable và tốc độ suy luận của từng cấu hình.

Các kỳ vọng trên là giả thuyết; báo cáo A1.2 sẽ đối chiếu với số liệu đo được.

3 Tập dữ liệu

3.1 Food-101

Food-101 [1] gồm 101 lớp món ăn, mỗi lớp 1 000 ảnh: 750 ảnh train và 250 ảnh test theo split chính thức. Ảnh test đã được kiểm tra thủ công. Ảnh train được các tác giả *cố ý* để nguyên, nên còn nhiễu, chủ yếu là màu quá gắt và đôi khi sai nhãn. Mọi ảnh đã được thu nhỏ để cạnh dài nhất không quá 512 px. Dữ liệu được tải qua `torchvision.datasets.Food101`.

3.2 Giấy phép sử dụng

Food-101 không có giấy phép chuẩn: trường `license` trong dataset card của bộ dữ liệu trên Hugging Face (`ethz/food101`) ghi `unknown`, và trang chủ của ETH Zürich không nêu điều khoản sử dụng. Mục *Licensing Information* của dataset card đó cho biết ảnh lấy từ Foodspotting và không thuộc sở hữu của ETH Zürich; mọi cách dùng vượt quá *scientific fair use* phải thoả thuận với chủ sở hữu từng ảnh theo điều khoản của Foodspotting. Bài tập chỉ dùng tập dữ liệu cho học tập và nghiên cứu, không phân phối lại ảnh: thư mục `data/` không được đưa lên repo.

3.3 Lý do chọn

Đề yêu cầu một tập ảnh công khai, tối thiểu 10 lớp và từ vài nghìn ảnh train trở lên. Food-101 được chọn theo bốn tiêu chí:

- **Đủ lớn và đủ khó.** 101 lớp, 75 750 ảnh train chính thức. Bài toán thuộc loại phân loại chi tiết (*fine-grained*): cùng một món có thể trông rất khác nhau, còn nhiều món khác nhau lại trông gần giống nhau.
- **Không trùng ảnh với ImageNet.** Tiny-ImageNet, Imagenette và các tập con của ImageNet bị loại, vì ảnh của chúng lấy từ ImageNet và phần lớn nằm trong tập train mà mô hình pretrained đã học, khiến so sánh scratch với pretrained bị lệch.
- **Hợp với mô hình pretrained.** Ảnh tự nhiên, cỡ vài trăm pixel, dùng thẳng ở độ phân giải 224 px. CIFAR-100 (ảnh 32×32) phải phóng to gấp 7 lần nên bị loại.
- **Vừa ngân sách.** Với khoảng 68 000 ảnh train ở 224 px, ước tính (chưa đo) cho thấy train từ đầu vài chục epoch trên một GPU L4 vẫn nằm trong ngân sách (Mục 11). Oxford Flowers-102 quá nhỏ (10 ảnh train mỗi lớp) để chế độ scratch có ý nghĩa; Stanford Cars bị loại vì torchvision không còn tải được tập này: đường dẫn gốc của Stanford đã hỏng và hàm tải báo lỗi, nên phải dựng dữ liệu thủ công từ nguồn khác.

Một điểm cần lưu ý khi phân tích. ImageNet-1k có sẵn một số lớp món ăn trùng hoặc rất gần với lớp của Food-101 (tên ImageNet ↔ tên Food-101): pizza ↔ `pizza`, hotdog ↔ `hot_dog`, ice cream ↔ `ice_cream`, guacamole ↔ `guacamole`, cheeseburger ↔ `hamburger`, carbonara ↔ `spaghetti_carbonara`, burrito ↔ `breakfast_burrito`. Ảnh khác nhau, nhưng khái niệm trùng hoặc gần nhau, nên trọng số pretrained có lợi thế ngữ nghĩa ở các lớp này. Báo cáo A1.2 sẽ xem accuracy từng lớp có phản ánh điều đó không.

4 Chia train, validation, test

Split test chính thức được giữ nguyên. Validation được tách từ split train chính thức: 10% mỗi lớp, phân tầng theo nhãn (`train_test_split(stratify=...)`, seed 42). Bảng 2 ghi số ảnh của từng tập.

Bảng 2: Ba tập dùng trong A1. Mọi lớp có cùng số ảnh trong mỗi tập.

Tập	Số ảnh	Ảnh/lớp	Nguồn
train	68 175	675	90% split train chính thức
validation	7 575	75	10% split train chính thức, phân tầng, seed 42
test	25 250	250	split test chính thức (đã làm sạch)

Lý do của từng lựa chọn:

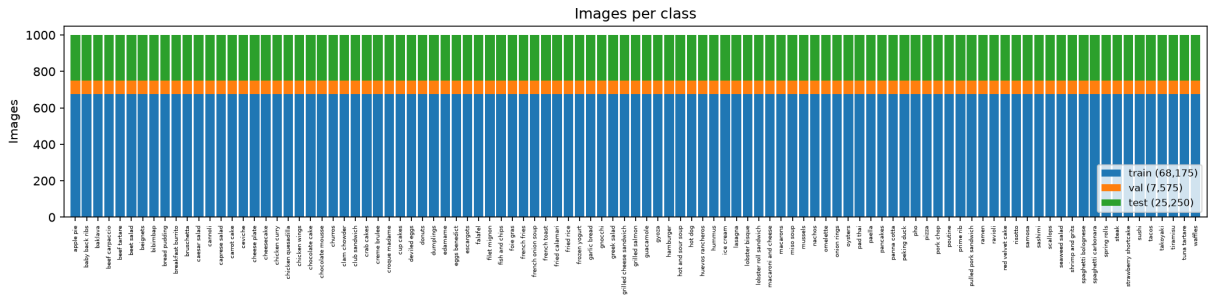
- **Test chỉ dùng một lần.** Mọi quyết định (epoch giữ lại, learning rate) dựa trên validation. Test được đánh giá đúng một lần cho mỗi run, sau khi train xong, để con số báo cáo không bị lạc quan do chọn mô hình trên chính tập báo cáo. Giữ split test chính thức còn giúp so sánh được với các công trình khác trên Food-101.
- **Validation lấy từ train, không lấy từ test.** Tách từ test sẽ vừa thu nhỏ tập báo cáo, vừa để một phần của nó tham gia chọn mô hình.
- **Tỉ lệ 10%.** Với accuracy khoảng 85%, sai số chuẩn trên 7 575 ảnh validation là $\sqrt{0.85 \cdot 0.15/7575} \approx 0.41\%$, tức khoảng ± 0.8 điểm ở mức tin cậy 95%; trên 25 250 ảnh test là khoảng ± 0.44 điểm. Lấy nhiều hơn sẽ bớt ảnh train, trong khi mỗi lớp chỉ có 750 ảnh.
- **Seed của split tách khỏi seed huấn luyện.** Split luôn dùng seed 42, nên mọi run, ở cả hai họ và cả bốn chế độ, dùng cùng một split, đúng yêu cầu "cùng split" của đề.

Validation lấy từ split train nên cũng mang nhiễu nhãn, trong khi test đã được kiểm tra thủ công; validation accuracy vì vậy có thể thấp hơn test accuracy. Mọi cấu hình chịu cùng mức nhiễu, nên việc xếp hạng và chọn checkpoint trên validation vẫn dùng được.

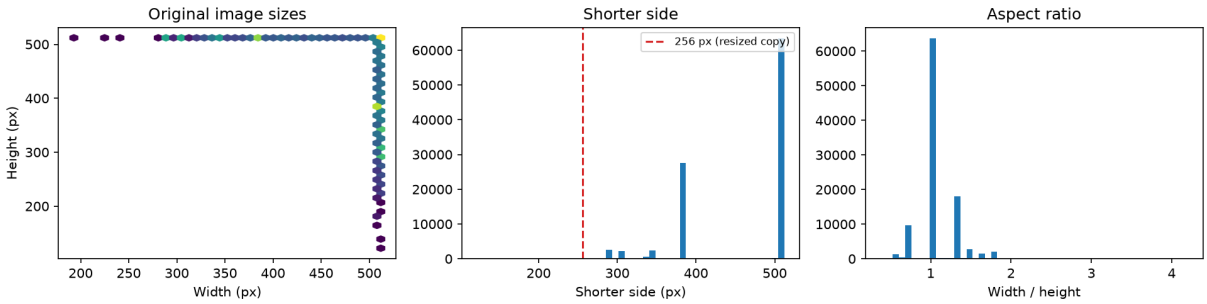
Ngoài ba tập trên, evaluator `train_eval` đọc một tập con phân tầng cố định gồm 10% train (6 868 ảnh, biến đổi như lúc đánh giá) để vẽ đường accuracy trên train mỗi epoch mà không phải duyệt hết 68 175 ảnh. Danh sách id ảnh của từng tập được ghi ra `data/food101_256/splits/*.csv` để tái lập.

5 Khảo sát dữ liệu

`scripts/a1/eda.py` sinh các hình và bảng dưới đây vào `docs/assets/a1/eda/`: số ảnh mỗi lớp trong từng tập (Hình 1), phân bố kích thước ảnh gốc (Hình 2), ảnh mẫu (Hình 3) và thống kê tổng hợp (Bảng 3). Kích thước được đo trên toàn bộ 101 000 ảnh gốc, trước khi thu nhỏ. Mean và std theo kênh được tính trên toàn bộ 68 175 ảnh train, sau bước cắt khi đánh giá.



Hình 1: Số ảnh mỗi lớp trong train, validation và test.



Hình 2: Kích thước ảnh gốc: phân bố rộng–cao, cạnh ngắn (vạch đỏ là 256 px của bản đã thu nhỏ) và tỉ lệ khung.

Bảng 3: Thống kê ảnh gốc và mean/std theo kênh RGB.

Đại lượng	Giá trị
Ảnh gốc	101 000 (L 3, RGB 100 997)
Cạnh dài (px)	512–512, trung vị 512
Cạnh ngắn (px)	122–512, trung vị 512
Cạnh ngắn < 256 px	44 ảnh
Tỉ lệ rộng/cao	0.38–4.20, trung vị 1.00
Mean RGB (68 175 ảnh train)	(0.557, 0.442, 0.327)
Std RGB	(0.260, 0.264, 0.266)



Hình 3: Ảnh train của một số lớp, sau bước Resize(256) + CenterCrop(224) dùng khi đánh giá. Bên trái: bốn lớp đầu; bên phải: bốn lớp sau.

Nhận xét.

- **Cân bằng lớp.** Mỗi lớp có đúng 675 ảnh train, 75 ảnh validation và 250 ảnh test (Hình 1). Dữ liệu cân bằng hoàn toàn nên không cần trọng số lớp hay lấy mẫu lại; vì mỗi lớp test có cùng số ảnh, accuracy trên test đúng bằng trung bình recall của 101 lớp.
- **Kích thước ảnh.** Cạnh dài luôn là 512 px. Theo Hình 2, khoảng 63 nghìn ảnh (63%) là ảnh vuông 512×512, khoảng 27 nghìn ảnh (27%) có cạnh ngắn 384 px (khung 4:3, khoảng 18 nghìn ảnh ngang và 9.5 nghìn ảnh dọc), phần còn lại rải ở các cạnh ngắn 288–345 px và các tỉ lệ khác. Các tỉ lệ này đọc từ biểu đồ nên chỉ xấp xỉ. Chỉ 44 ảnh (0.04%) có cạnh ngắn dưới 256 px: bước tiền xử lý giữ nguyên chúng, và `Resize(256)` lúc đánh giá phóng chúng lên; số ảnh này quá ít để ảnh hưởng đến kết quả. Tỉ lệ khung lớn nhất là 4.2, nhưng ảnh khung dài rất hiếm; với chúng, `CenterCrop(224)` chỉ giữ phần giữa và bỏ phần lớn ảnh.
- **Chế độ màu.** 100 997 ảnh RGB, 3 ảnh xám và không có ảnh CMYK. Ba ảnh xám được chuyển sang RGB ở bước tiền xử lý nên không cần xử lý riêng.
- **Màu sắc.** Mean RGB trên toàn bộ ảnh train là $(0.557, 0.442, 0.327)$, std khoảng 0.26 ở cả ba kênh. So với ImageNet $(0.485, 0.456, 0.406)$, ảnh món ăn nghiêng nhiều về đỏ và ít xanh dương hơn. Sau phép chuẩn hoá mean = std = 0.5 của pipeline, mean các kênh còn khoảng $(+0.11, -0.12, -0.35)$ và std khoảng 0.52, tức nhỏ hơn hai lần so với khi pretrain; điều này nhất quán với lưu ý về chuẩn hoá ở Mục 6.
- **Biến thiên trong lớp và nhiễu nhãn** (Hình 3). Ảnh chủ yếu chụp bằng điện thoại trong nhà hàng: ánh sáng và góc chụp rất khác nhau (nhiều ảnh tối hoặc dùng đèn flash), nền lộn xộn và thường có nhiều món trong cùng một khung. Trong 48 ảnh mẫu, ít nhất hai ảnh mang nhãn không ứng với chủ thể rõ ràng: ảnh thứ hai của *bibimbap* (chủ yếu là há cảo và các đĩa phụ) và ảnh thứ ba của *grilled cheese sandwich* (khoai chiên và rau, không thấy bánh sandwich). Một vài ảnh có màu rất gắt, ví dụ ảnh thứ tư của *beef tartare* và ảnh thứ hai của *prime rib*. Đây đúng là hai loại nhiễu mà các tác giả mô tả ở tập train. Mẫu 48 ảnh quá nhỏ để ước lượng tỉ lệ nhiễu.
- **Cặp lớp dễ nhầm** (dự kiến, chưa kiểm tra). Food-101 có nhiều món gần nhau về hình thức, ví dụ *prime rib*, *steak* và *filet mignon*, hay các món súp kem như *lobster bisque* và *clam chowder*. Các cặp thực sự bị nhầm sẽ được đo ở A1.2 bằng ma trận nhầm lẫn (Mục 10).

Hai hệ quả cho giao thức: validation lấy từ split train nên mang cùng loại nhiễu và có thể cho accuracy thấp hơn test; và các ảnh nhiễu sẽ được tách khỏi lõi thật của mô hình bằng lưới ảnh bị đoán sai với độ tự tin cao nhất (Mục 10).

6 Tiền xử lý và tăng cường dữ liệu

Giống MNIST ở E1 và E2, **mọi mô hình và mọi chế độ dùng chung một pipeline dữ liệu**. Các run chỉ khác nhau ở mô hình và cách huấn luyện, không khác nhau ở dữ liệu đưa vào.

Tiền xử lý một lần (`scripts/a1/prepare_data.py`). Ảnh được chuyển sang RGB, cạnh ngắn được thu về 256 px (giữ tỉ lệ, nội suy bicubic, ảnh vốn nhỏ hơn thì giữ nguyên) và lưu JPEG chất lượng 90 vào `data/food101_256/`. Mục đích là tốc độ: giải mã ảnh gốc cạnh dài 512 px ở tốc độ train tốn nhiều CPU hơn, và nếu DataLoader không theo kịp thì GPU phải chờ dữ liệu; tốc độ thực tế sẽ được đo trước khi chạy chính thức. Ảnh 256 px vẫn đủ cho cả hai phép cắt 224 px ở Bảng 4.

Các phép biến đổi hình học ở mức vừa phải. `RandomResizedCrop` chỉ cắt 80–100% diện tích, nên món ăn hầu như luôn còn trong khung. Phép xoay $\pm 15°$ và phép dịch tối đa 10% để lại các góc tô đen (giá trị 0), thấy rõ ở Hình 4. `ColorJitter` đổi độ sáng, tương phản và độ bão hoà trong khoảng $\pm 20\%$, và xoay sắc độ (`hue`) tới ± 0.1 vòng màu, tức tới $\pm 36°$. Mức xoay sắc độ này không nhỏ: đủ để đổi đỏ sang cam hay tím. Màu sắc lại là đặc trưng quan trọng của món ăn, nên đây là phép biến đổi mạnh nhất của pipeline và cần được theo dõi khi phân tích lỗi.

Bảng 4: Pipeline dữ liệu dùng chung, theo đúng thứ tự áp dụng (`TRAIN_STEPS`, `EVAL_STEPS` trong `scripts/a1/make_configs.py`).

Bước	Train	Đánh giá (validation, test, train_eval)
Cắt	`RandomResizedCrop(224, scale=(0.8, 1.0))`	`Resize(256)`, `CenterCrop(224)`
Lật	`RandomHorizontalFlip(p=0.5)`	–
Xoay	`RandomRotation(degrees=15)`	–
Màu	`ColorJitter(0.2, 0.2, 0.2, hue=0.1)`	–
Dịch	`RandomAffine(degrees=0, translate=(0.1, 0.1))`	–
Chuẩn hoá	`ToTensor`, `Normalize`(mean 0.5, std 0.5)	như train



Hình 4: Cột đầu: ảnh sau bước cắt khi đánh giá. Các cột sau: cùng ảnh qua pipeline train sáu lần.

Chuẩn hoá và trọng số pretrained. Pipeline chuẩn hoá với mean = std = 0.5 cho cả ba kênh, đưa ảnh về khoảng $[-1, 1]$. Cả hai bộ trọng số pretrained được huấn luyện với mean $(0.485, 0.456, 0.406)$ và std $(0.229, 0.224, 0.225)$ của ImageNet, nên đầu vào lệch so với lúc pretrain: std nhỏ hơn khoảng 2.2 lần. Chế độ full finetune được kỳ vọng tự thích nghi. Chế độ head và partial giữ cố định phần mạng bị đóng băng, kể cả thống kê của BatchNorm ở ResNet-50, nên có thể chịu thiệt. Đây là đánh đổi có chủ đích để có một pipeline chung, và cần được tính đến khi so sánh head, partial với full.

Nạp dữ liệu. DataLoader train dùng batch 128, xáo trộn, bỏ batch cuối thiếu (`drop_last`), 8 worker; hai loader đánh giá dùng batch 256 và 4 worker. Cả ba loader dùng `pin_memory` và giữ worker giữa các epoch.

7 Mô hình

Hai mô hình lấy từ thư viện timm [6] (bản 1.0.30), có số tham số và chi phí tính toán gần nhau, và cùng được pretrain chỉ trên ImageNet-1k, để nguồn pretrain không làm lệch phép so sánh (Bảng 5). Head phân loại được thay bằng một lớp tuyến tính 101 lớp, khởi tạo ngẫu nhiên.

Bảng 5: Hai mô hình. Số tham số tính với head 101 lớp. MAC cho một ảnh 224×224, đếm bằng `FlopCounterMode` với attention tính bằng phép nhân ma trận thường (`timm.layers.set_fused_attn(False)`): bộ đếm không ghi nhận kernel attention gộp (SDPA), nên để mặc định sẽ thiếu khoảng 0.36 GMAC của DeiT-S.

Họ	Trọng số timm	Tham số	GMAC/ảnh	Pretrain
CNN	`resnet50.a1_in1k`	23 714 981	4.09	ImageNet-1k
Transformer	`deit_small_patch16_224.fb_in1k`	21 704 549	4.60	ImageNet-1k

Kích thước đầu ra và số tham số trong hai mục dưới đây được đo trực tiếp trên mô hình timm (head 101 lớp, ảnh 224×224); tổng từng phần khớp đúng tổng số tham số ở Bảng 5.

7.1 ResNet-50

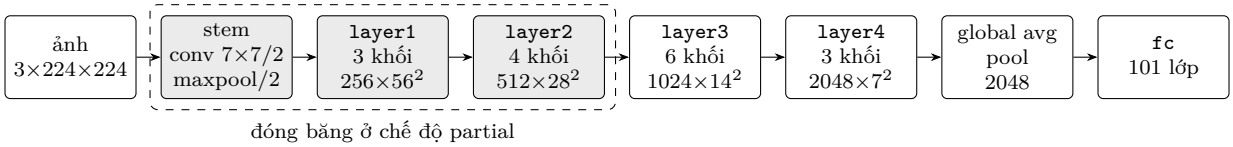
ResNet-50 [2] gồm một stem và bốn tầng khối *bottleneck* có kết nối tắt, rồi global average pooling và một lớp tuyến tính (Bảng 6, Hình 5). Mỗi phép tích chập đi kèm BatchNorm; hàm kích hoạt là ReLU (σ). Một khối bottleneck với c kênh ở giữa và bước nhảy s tính

$$\begin{aligned} F(x) &= \mathrm{BN}\big(W_3 * \sigma(\mathrm{BN}(W_2 *_s \sigma(\mathrm{BN}(W_1 * x))))\big), \\ y &= \sigma\big(F(x) + P(x)\big), \end{aligned} \tag{1}$$

trong đó W_1 là tích chập 1×1 thu số kênh về c, W_2 là tích chập 3×3 giữ c kênh với bước nhảy s, W_3 là tích chập 1×1 mở rộng lên $4c$ kênh. Nhánh tắt P là ánh xạ đồng nhất, trừ khối đầu của mỗi tầng: ở đó số kênh hoặc độ phân giải thay đổi nên P là tích chập 1×1 bước nhảy s kèm BatchNorm. Bước nhảy 2 đặt ở tích chập 3×3 (biến thể thường gọi là ResNet v1.5, mặc định trong timm và torchvision), không ở tích chập 1×1 đầu khối như bài báo gốc.

Bảng 6: Kiến trúc ResNet-50. Mỗi dòng "$[\cdot] \times n$" là n khối bottleneck; bước nhảy 2 ở khối đầu của tầng. Phần tô xám là phần bị đóng băng ở chế độ partial.

Tầng	Cấu hình	Đầu ra ($C \times H \times W$)	Tham số
stem	conv 7×7, 64, stride 2; maxpool 3×3, stride 2	$64 \times 56 \times 56$	9 536
`layer1`	$[1{\times}1, 64;\ 3{\times}3, 64;\ 1{\times}1, 256] \times 3$	$256 \times 56 \times 56$	215 808
`layer2`	$[1{\times}1, 128;\ 3{\times}3, 128;\ 1{\times}1, 512] \times 4$	$512 \times 28 \times 28$	1 219 584
`layer3`	$[1{\times}1, 256;\ 3{\times}3, 256;\ 1{\times}1, 1024] \times 6$	$1024 \times 14 \times 14$	7 098 368
`layer4`	$[1{\times}1, 512;\ 3{\times}3, 512;\ 1{\times}1, 2048] \times 3$	$2048 \times 7 \times 7$	14 964 736
head	global average pooling; `fc` $2048 \to 101$	101	206 949
tổng			23 714 981



Hình 5: Luồng dữ liệu qua ResNet-50: độ phân giải giảm từ 224 xuống 7 trong khi số kênh tăng từ 64 lên 2048.

Tham số dồn về cuối mạng: `layer3` và `layer4` chứa 22 063 104 tham số, tức 93.0% toàn mô hình, trong khi stem, `layer1` và `layer2` chỉ có 1 444 928 (6.1%). Chi phí tính toán thì chia khá đều giữa các tầng, vì mỗi lần độ phân giải giảm một nửa thì số kênh tăng gấp đôi: đo bằng `FlopCounterMode`, stem, `layer1`–`layer4` lần lượt chiếm 2.9, 16.3, 25.1, 35.8 và 19.8% MAC. Nửa đầu mạng (stem, `layer1`, `layer2`) vì vậy chiếm 44% MAC nhưng chỉ 6.1% tham số; đóng băng phần này ở chế độ partial bỏ được phần lan truyền ngược qua gần nửa khối lượng tính toán, dù

gần như không giảm số tham số phải học. Trọng số `a1_in1k` được train theo công thức mạnh của timm [5], cho top-1 trên ImageNet-1k ở 224×224 là 80.38%, gần với 79.86% của DeiT-S (bảng kết quả của timm), thay vì bản ResNet-50 gốc của torchvision, vốn thấp hơn.

7.2 DeiT-S

DeiT-S [4] có cùng kiến trúc với ViT-S/16 [3]; khác biệt của DeiT nằm ở công thức huấn luyện trên ImageNet-1k. Ảnh được cắt thành $14 \times 14 = 196$ patch 16×16; mỗi patch ($16 \cdot 16 \cdot 3 = 768$ giá trị) được chiếu tuyến tính lên $D = 384$ chiều, tương đương một tích chập 16×16 bước nhảy 16. Thêm token CLS ở đầu và cộng positional embedding học được, chuỗi gồm $N = 197$ token đi qua 12 khối Transformer *pre-norm*:

$$\begin{aligned} z_0 &= [\,x_{\text{cls}};\; x_p^1 E;\; \ldots;\; x_p^{196} E\,] + E_{\text{pos}}, \quad E \in \mathbb{R}^{768\times 384},\; E_{\text{pos}} \in \mathbb{R}^{197\times 384}, \\ z'_\ell &= z_{\ell-1} + \text{MSA}\big(\text{LN}(z_{\ell-1})\big), \qquad \ell = 1, \ldots, 12, \\ z_\ell &= z'_\ell + \text{MLP}\big(\text{LN}(z'_\ell)\big), \\ \hat{y} &= W_{\text{head}}\, \text{LN}\big(z_{12}^0\big) + b_{\text{head}}, \end{aligned} \tag{2}$$

với z_{12}^0 là token CLS sau khối cuối. MSA có $h = 6$ head, mỗi head $d_k = D/h = 64$ chiều:

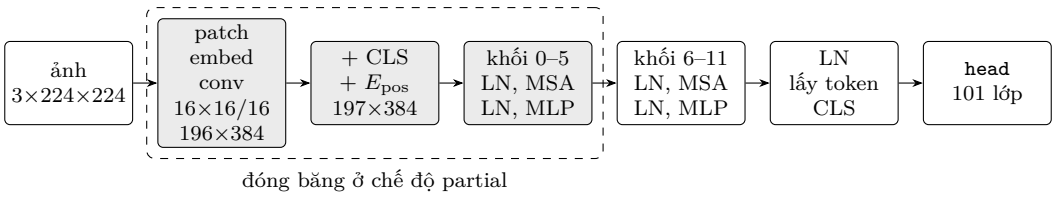
$$\text{head}_i = \text{softmax}\!\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i, \qquad \text{MSA}(X) = [\text{head}_1; \ldots; \text{head}_6]\, W_O, \tag{3}$$

trong đó Q, K, V lấy từ một phép chiếu chung $384 \to 3 \cdot 384$. MLP là `Linear` $384 \to 1536$, GELU, `Linear` $1536 \to 384$.

Bảng 7: Kiến trúc DeiT-S. Phần tô xám là phần bị đóng băng ở chế độ partial.

Thành phần	Cấu hình	Đầu ra ($N \times D$)	Tham số
patch embedding	conv 16×16, stride 16, $3 \to 384$	196×384	295 296
CLS, positional emb.	token 1×384; $E_{\text{pos}} \in \mathbb{R}^{197\times 384}$	197×384	76 032
khối 0–5	$6\times$ [LN; MSA 6 head $\times$ 64; LN; MLP $384 \to 1536 \to 384$]	197×384	10 646 784
khối 6–11	như trên	197×384	10 646 784
head	LN; `Linear` $384 \to 101$ trên token CLS	101	39 653
tổng			21 704 549

Mỗi khối có 1 774 464 tham số: MSA 591 360 (phép chiếu Q, K, V 443 520 và W_O 147 840), MLP 1 181 568, hai LayerNorm 1 536. Với $N = 197$, hai phép nhân ma trận của attention ($QK^\top$ và nhân với V) chỉ chiếm khoảng 7.9% MAC của một khối; phần lớn chi phí nằm ở các phép chiếu tuyến tính và MLP.



Hình 6: Luồng dữ liệu qua DeiT-S: 197 token $\times$ 384 chiều được giữ nguyên qua cả 12 khối.

Trọng số `fb_in1k` được train chỉ trên ImageNet-1k và là bản DeiT không có token distillation.

7.3 Khác biệt kiến trúc liên quan đến các câu hỏi của A1

- **Giả định cục bộ.** Tích chập 3×3 chỉ nhìn lân cận; vùng nhìn mở rộng dần qua độ sâu và các lần giảm độ phân giải. Attention nhìn toàn ảnh ngay từ khối đầu, và tính cục bộ chỉ

có nếu được học từ dữ liệu. Đây là cơ sở cho giả thuyết rằng DeiT-S cần pretrain nhiều hơn ResNet-50 (câu hỏi 1 và 2).

- **Phân bố tham số theo độ sâu.** ResNet-50 dồn 93% tham số vào hai tầng cuối, còn mười hai khối của DeiT-S có cùng kích thước. Vì vậy cùng là "đóng băng nửa đầu", ResNet-50 vẫn học 93.9% tham số còn DeiT-S học 49.2% (Bảng 8).
- **BatchNorm và LayerNorm.** BatchNorm của ResNet-50 dùng thống kê chạy tích luỹ khi pretrain, nên phần bị đóng băng giữ nguyên thống kê của ImageNet; khi pipeline chuẩn hoá khác lúc pretrain (Mục 6), sai lệch này không được sửa. LayerNorm của DeiT-S chuẩn hoá lại từng token theo chính nó, nên có thể bớt nhạy với thay đổi về thang đo của đầu vào. Đây là giả thuyết, sẽ được xem lại khi so sánh chế độ head và partial của hai mô hình.
- **Chi phí.** Hai mô hình gần nhau về số tham số (23.7 và 21.7 triệu) và MAC (4.09 và 4.60 GMAC). MAC của ResNet-50 chia khá đều giữa bốn tầng (nhiều nhất ở `layer3`, 35.8%), còn DeiT-S giữ 197 token suốt 12 khối nên MAC chia đều cho các khối. Ở chế độ partial, phần bị đóng băng chiếm 44% MAC của ResNet-50 và khoảng một nửa MAC của DeiT-S.

8 Chế độ huấn luyện

`models/a1/classifier.py` hiện thực bốn chế độ; Bảng 8 ghi định nghĩa và số tham số được học của mỗi mô hình ở từng chế độ.

Bảng 8: Bốn chế độ huấn luyện. "Nửa đầu" tính theo độ sâu: ResNet-50 đóng băng stem cùng `layer1`–`layer2`, DeiT-S đóng băng patch embedding, token CLS, positional embedding và khối 0–5.

Chế độ	Pretrained	Đóng băng	ResNet-50 trainable	DeiT-S trainable
scratch	không	không	23 714 981 (100%)	21 704 549 (100%)
head	có	mọi thứ trừ head	206 949 (0.9%)	38 885 (0.2%)
partial	có	nửa đầu	22 270 053 (93.9%)	10 686 437 (49.2%)
full	có	không	23 714 981 (100%)	21 704 549 (100%)

Bất đối xứng của chế độ partial. Cùng là "đóng băng nửa đầu theo độ sâu", nhưng ResNet-50 vẫn học 93.9% tham số, vì tham số của nó dồn ở `layer3` và `layer4`, trong khi DeiT-S chỉ học 49.2%, vì 12 khối của nó có cùng kích thước. Chúng tôi giữ định nghĩa theo độ sâu, vì nó ứng với câu hỏi "giữ nguyên các đặc trưng cấp thấp", và luôn báo cáo số tham số trainable bên cạnh kết quả để người đọc thấy rõ khác biệt này.

BatchNorm trong phần đóng băng. Đóng băng trọng số là chưa đủ: ở chế độ train, BatchNorm vẫn cập nhật trung bình và phương sai chạy. `A1Classifier.train()` đặt lại mọi lớp BatchNorm có toàn bộ tham số bị đóng băng về chế độ eval. Lần chạy thử cho thấy sau hai epoch ở chế độ head, mọi tensor của ResNet-50 ngoài `fc`, kể cả thống kê BatchNorm, giữ nguyên giá trị pretrained.

Weight decay và stochastic depth. Weight decay chỉ áp lên trọng số của các lớp tuyến tính và tích chập; bias, tham số chuẩn hoá, token CLS và positional embedding không bị decay. Stochastic depth (drop path) đặt 0.1 cho scratch và full, và 0 cho head và partial: bỏ ngẫu nhiên các khối đã đóng băng chỉ làm nhiễu đặc trưng cố định mà không giúp học thêm.

9 Giao thức thí nghiệm

Ma trận thí nghiệm gồm **8 run**: 2 mô hình × 4 chế độ, mỗi run train một lần với seed 42 trên toàn bộ tập train (Bảng 9). Tất cả cấu hình được sinh bởi `scripts/a1/make_configs.py` vào `configs/a1/`.

Bảng 9: Siêu tham số riêng theo chế độ, dùng cho cả hai mô hình. Learning rate là giá trị khởi đầu, sẽ được kiểm tra trên validation trước khi chạy chính thức.

Chế độ	Epoch	LR đỉnh	Warmup (epoch)	Drop path	Clip gradient
scratch	40	5×10^{-4}	5	0.1	1.0
head	10	1×10^{-3}	1	0	–
partial	15	2×10^{-4}	1	0	–
full	15	1×10^{-4}	1	0.1	–

Phần dùng chung cho mọi run: AdamW với weight decay 0.05, batch 128, cross-entropy với label smoothing 0.1, autocast bfloat16 trên GPU, và memory format `channels_last` cho ResNet-50. Learning rate tăng tuyến tính trong W bước đầu rồi giảm theo cosine về 0, cập nhật sau mỗi bước tối ưu:

$$\eta_t = \begin{cases} \eta_{\max} \dfrac{t+1}{W}, & t < W, \\ \eta_{\max} \dfrac{1}{2}\left(1 + \cos \dfrac{\pi\,(t-W)}{T-W}\right), & t \geq W, \end{cases} \tag{4}$$

với T là tổng số bước (số epoch $\times$ 532 bước mỗi epoch) và W là số epoch warmup $\times$ 532.

Chọn checkpoint và dừng. Sau mỗi epoch, mô hình được đánh giá trên validation, và checkpoint có validation accuracy cao nhất được giữ lại. Không dùng early stopping: lịch cosine cần chạy hết để learning rate về 0. Một bản sao lưu đầy đủ (mô hình, optimizer, scheduler, trạng thái ngẫu nhiên) được ghi mỗi epoch, nên run bị ngắt có thể chạy tiếp đúng từ chỗ dừng.

Ngân sách epoch. Scratch được 40 epoch, các chế độ pretrained 10–15 epoch. So sánh trong A1 vì vậy là so sánh *dưới ngân sách cố định*, không phải giữa các mô hình đã hội tụ hoàn toàn; thời gian train được báo cáo cùng accuracy để thấy rõ đánh đổi.

Kiểm tra learning rate. Trước khi chạy chính thức, learning rate của scratch và full được thử ngắn (2–3 epoch) trên validation. Tập test không được dùng ở bước này. Mỗi lần thử là một run riêng, ghi vào thư mục riêng và không bao giờ được dùng làm run chính.

10 Chiến lược đánh giá

Mục này cố định trước khi chạy: đo đại lượng nào, trên tập nào, so sánh những cặp nào, và khi nào một chênh lệch được coi là có thật. Mục đích là để kết luận ở A1.2 không phụ thuộc vào việc nhìn số rồi mới chọn cách đọc.

10.1 Nguyên tắc

- **Validation cho mọi quyết định, test đúng một lần.** Chọn epoch, kiểm tra learning rate và chọn mô hình để phân tích lỗi đều dựa trên validation. Mỗi run được đánh giá trên test một lần duy nhất, sau khi train xong.
- **Cùng điều kiện cho mọi run.** Cùng 25 250 ảnh test, cùng pipeline đánh giá (Bảng 4: `Resize(256)`, `CenterCrop(224)`, không tăng cường dữ liệu), mô hình ở chế độ eval (tắt dropout và drop path, BatchNorm dùng thống kê chạy), autocast bfloat16, logits được chuyển về float32 trước khi tính loss và các metric.
- **Một checkpoint cho mỗi run.** Giữ epoch có validation accuracy cao nhất; nếu bằng nhau thì lấy epoch sớm hơn. Val acc và Test acc trên cùng một dòng của bảng kết quả vì vậy thuộc về cùng một mô hình.
- **Test không dùng để chọn.** Không chọn learning rate, epoch hay "mô hình tốt nhất" dựa trên test. Hai mô hình đem đi phân tích lỗi (một CNN, một Transformer) được chọn theo validation.

10.2 Đại lượng đo

Bảng 10: Các đại lượng báo cáo cho mỗi run và cách đo.

Nhóm	Đại lượng	Cách đo
Chất lượng	test accuracy (top-1)	đại lượng chính; từ ma trận nhầm lẫn 101×101 trên 25 250 ảnh test
	precision, recall, F1	trung bình macro trên 101 lớp, mỗi lớp trọng số như nhau
	validation accuracy	tại epoch được giữ; so với test để thấy ảnh hưởng của nhiễu nhãn
	accuracy từng lớp	đường chéo của ma trận nhầm lẫn chia cho số ảnh của lớp
Chi phí	tham số tổng và trainable	đếm từ mô hình đã áp chế độ đóng băng
	thời gian mỗi epoch	chỉ các bước train, không tính thời gian validation (`EpochTimer`)
	bộ nhớ GPU đỉnh khi train	`torch.cuda.max_memory_allocated` trong các bước train mỗi epoch
	tốc độ suy luận (ảnh/giây)	một batch test đặt sẵn trên GPU, chạy lặp sau khi khởi động; không phụ thuộc DataLoader
Lỗi	cặp lớp bị nhầm nhiều nhất	15 cặp (lớp thật $\rightarrow$ lớp đoán) có số ảnh nhầm lớn nhất
	ảnh sai với độ tự tin cao	các ảnh bị đoán sai mà softmax tối đa lớn nhất
	dự đoán từng ảnh test	để so sánh cùng một ảnh giữa các run

10.3 Độ tin cậy của kết quả với một seed

Mỗi cấu hình chỉ chạy một lần (seed 42), nên không đo được độ dao động giữa các lần train. Báo cáo vì vậy tách hai nguồn sai số và chỉ định lượng được nguồn thứ nhất:

- **Sai số do chọn tập test hữu hạn.** Với accuracy khoảng 85%, khoảng tin cậy Wilson 95% của một run trên 25 250 ảnh là khoảng ± 0.44 điểm (trên 7 575 ảnh validation là khoảng ± 0.8 điểm). Mỗi run được báo cáo kèm khoảng này.
- **Sai số do lần train cụ thể** (khởi tạo, thứ tự dữ liệu, augmentation). Không đo được ở đây và không có phép kiểm định nào thay thế được; đây là lý do ngưỡng ở dưới đòi hỏi một chênh lệch tối thiểu.

Hai run được chấm trên *cùng* các ảnh test, nên chênh lệch giữa chúng được đánh giá theo cặp, chặt hơn so với so sánh hai khoảng tin cậy riêng lẻ. Ví dụ, nếu hai mô hình bất đồng trên khoảng 8% số ảnh thì khoảng tin cậy 95% của chênh lệch accuracy chỉ rộng khoảng ± 0.35 điểm. Với mỗi so sánh sẽ dùng:

- khoảng tin cậy 95% bằng bootstrap theo cặp: lấy mẫu có hoàn lại các ảnh test, chấm mọi run trên cùng một mẫu, 2 000 lần;
- kiểm định McNemar chính xác (hai phía) trên các ảnh mà đúng một trong hai run đoán đúng, với hiệu chỉnh Holm cho cả nhóm so sánh dưới đây.

Quy tắc kết luận, cố định trước khi có số liệu:

- *rõ ràng*: khoảng tin cậy không chứa 0 **và** chênh lệch tối thiểu 1 điểm accuracy;
- *nhỏ*: khoảng tin cậy không chứa 0 nhưng chênh lệch dưới 1 điểm; chỉ mô tả, không dùng làm luận cứ, vì cỡ này nằm trong mức một seed khác có thể làm thay đổi;
- *chưa kết luận được*: khoảng tin cậy chứa 0.

Ngưỡng 1 điểm là một lựa chọn thận trọng, không rút ra từ dữ liệu. Nếu thời gian cho phép, chạy thêm seed cho vài cấu hình chính sẽ thay ngưỡng này bằng một độ dao động đo được.

10.4 Các so sánh được định trước

Bảng 11: Mười một so sánh theo kế hoạch, gắn với bốn câu hỏi ở Mục 2. Kiểm định McNemar áp dụng cho mười cặp đầu; so sánh thứ mười một (chênh của hai chênh lệch) chỉ có khoảng bootstrap.

Câu hỏi	So sánh	Cách đọc
1. CNN hay ViT	DeiT-S − ResNet-50, trong mỗi chế độ (scratch, head, partial, full): 4 cặp	chênh dương: Transformer tốt hơn
2. Giá trị của pretrain	full − scratch, riêng từng mô hình: 2 cặp; và (DeiT-S full − scratch) − (ResNet-50 full − scratch)	chênh dương ở so sánh cuối: pretrain giúp Transformer nhiều hơn CNN
3. Giá của việc đóng băng	head − full và partial − full, riêng từng mô hình: 4 cặp	chênh âm: accuracy mất đi khi đóng băng
4. Chi phí	không kiểm định; đọc Bảng 10 và biểu đồ accuracy theo thời gian train	accuracy đổi lấy thời gian, bộ nhớ, tham số

Chỉ các so sánh này được kiểm định. Mọi so sánh khác (ví dụ head với partial) chỉ mô tả, và được ghi rõ là phát sinh sau khi xem kết quả.

10.5 Điều kiện công bằng và hạn chế khi đọc kết quả

- **Ngân sách epoch khác nhau giữa các chế độ** (40 cho scratch, 10–15 cho các chế độ pretrained). So sánh *trong* một chế độ giữa hai mô hình là cùng ngân sách; so sánh scratch với full thì không. Điều này thiên lệch theo hướng thận trọng: scratch được nhiều epoch hơn, nên khoảng cách đo được không phóng đại giá trị của pretrain.
- **Thời gian mỗi epoch dao động giữa các run**, vì mỗi run chạy trong một container riêng, có thể khác máy và khác thời điểm, và bao gồm cả thời gian DataLoader. Để so sánh tốc độ giữa các mô hình sẽ dùng kết quả đo của `benchmark.py` (mọi mô hình đo tuần tự trên cùng một GPU, dữ liệu sinh sẵn trên GPU) và tốc độ suy luận; cột thời gian mỗi epoch chỉ để tham khảo.
- **Nhiễu nhãn.** Validation lấy từ split train nên nhiễu, trong khi test đã được làm sạch; validation accuracy có thể thấp hơn test accuracy. Mọi cấu hình chịu cùng mức nhiễu, nên việc chọn checkpoint và xếp hạng trên validation vẫn dùng được, nhưng độ lớn tuyệt đối của hai con số không so với nhau.
- **Chuẩn hoá khác lúc pretrain** (Mục 6) có thể làm thấp kết quả của head và partial; các so sánh ở nhóm câu hỏi 3 cần được đọc kèm điều này.
- **Trùng khái niệm với ImageNet.** Accuracy từng lớp sẽ được dùng để xem phần lợi của pretrain có tập trung ở các lớp món ăn vốn có trong ImageNet hay không, thay vì chỉ nhìn con số trung bình.

10.6 Phân tích lỗi

Sau khi có số liệu, `scripts/a1/analyze.py` lấy checkpoint tốt nhất của từng run, đánh giá một lần trên test và ghép với log huấn luyện có cùng dấu thời gian. Phân tích lỗi dùng mô hình CNN và mô hình Transformer có validation accuracy cao nhất:

- **Cặp lớp bị nhầm nhiều nhất**, để xem lỗi dồn vào các món trông giống nhau hay rải đều.
- **Lưới ảnh bị đoán sai với độ tự tin cao nhất.** Giúp tách nhầm lẫn giữa các món gần giống nhau khỏi các ảnh mà nhãn không đáng tin; lỗi thuộc loại sau là giới hạn của dữ liệu, không phải của mô hình.
- **Dự đoán từng ảnh test**, để xem CNN và Transformer sai trên cùng những ảnh hay trên những ảnh khác nhau.

Kết quả gồm `results_a1.csv`, bảng LaTeX `results_a1.tex`, đường validation accuracy theo epoch của bốn chế độ (`curves_a1.png`), biểu đồ accuracy theo thời gian train (`cost_a1.png`), các hình `confused_<run>.png`, `errors_<run>.png`, và dự đoán từng ảnh trong `predictions/`.

11 Hạ tầng, tái lập và ngân sách

Mã nguồn. `datasets/food101.py` kế thừa `torchvision.datasets.Food101` và thêm split validation cùng tập con phân tầng, giống cách `datasets/mnist.py` kế thừa MNIST. `models/a1/classifier.py` dựng mô hình timm với ba chế độ đóng băng. Vòng lặp huấn luyện, metric, checkpoint và log TensorBoard dùng lại khung chung trong `common/` từ E1 và E2, có thêm autocast bfloat16, scheduler theo từng bước và đo bộ nhớ đỉnh. Các script của A1 nằm trong `scripts/a1/`, hướng dẫn chạy ở `configs/a1/README.md`.

Tái lập. Toàn bộ config được sinh từ một file; split cố định và có danh sách id ảnh; seed 42 được đặt trước khi dựng mô hình; phiên bản thư viện được ghim trong `requirements.txt`.

Phần cứng và ngân sách. Thí nghiệm chạy trên Modal, mỗi run là một container với một GPU NVIDIA L4 (24 GB), 8 nhân CPU và 16 GB RAM. Ngân sách tối đa là \$20, với đơn giá ước tính khoảng \$1.3–1.6 mỗi giờ cho GPU, CPU và bộ nhớ. Mốc A1.1 chưa có số đo tốc độ. Ước tính sơ bộ dựa trên giả định khoảng 800 ảnh/giây khi train (chưa kiểm chứng) là khoảng 4 GPU-giờ cho 8 run, tức khoảng \$6; nếu tốc độ thực chỉ bằng một nửa thì khoảng \$10–13, vẫn trong ngân sách. Trước khi chạy chính thức, `scripts/a1/benchmark.py` sẽ đo tốc độ train, tốc độ suy luận và bộ nhớ đỉnh của từng mô hình ở từng chế độ trên chính GPU đó, rồi số epoch ở Bảng 9 được chốt theo kết quả đo và ghi vào báo cáo A1.2.

12 Kế hoạch, rủi ro và hạn chế

12.1 Kế hoạch

Thời gian	Công việc
17–19/10	Khung mã: dữ liệu, mô hình, cấu hình, phân tích, chạy thử trên dữ liệu giả; tải, thu nhỏ Food-101 và EDA trên toàn bộ dữ liệu (Modal)
20–23/10	Benchmark trên GPU L4 để chốt số epoch; kiểm tra learning rate trên validation
24–26/10	Hoàn thiện và nộp A1.1; khởi động hai run scratch
27/10–03/11	Sáu run còn lại; đánh giá trên test
04–09/11	Phân tích, báo cáo A1.2, trang A1 trên GitHub Pages

12.2 Rủi ro và hạn chế

- **Một seed.** Mỗi cấu hình chỉ chạy một lần, nên không có khoảng dao động giữa các lần chạy. Chênh lệch dưới khoảng 1 điểm accuracy trên test sẽ không được coi là khác biệt.
- **Ngân sách epoch ngắn cho scratch.** 40 epoch có thể chưa đủ để DeiT-S hội tụ khi train từ đầu. Kết luận về scratch là kết luận dưới ngân sách này.
- **Chuẩn hoá khác lúc pretrain** (Mục 6) có thể làm head và partial thấp hơn khả năng thật.
- **Nhiễu nhãn** trong train và validation làm validation accuracy thấp hơn test và giới hạn accuracy đạt được.
- **Lớp trùng khái niệm với ImageNet** (Mục 3.3) có thể làm phóng đại lợi ích của pretrain ở một số lớp.

Tài liệu tham khảo

[1] L. Bossard, M. Guillaumin, L. Van Gool. Food-101 – Mining Discriminative Components with Random Forests. *ECCV*, 2014.

[2] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[3] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.

[4] H. Touvron et al. Training data-efficient image transformers & distillation through attention. *ICML*, 2021.

[5] R. Wightman, H. Touvron, H. Jégou. ResNet strikes back: An improved training procedure in timm. *arXiv:2110.00476*, 2021.

[6] R. Wightman. PyTorch Image Models (timm). `https://github.com/huggingface/pytorch-image-models`, 2019.